

HIGH-SCALE E-COMMERCE SALE

Distributed High-Throughput System Architecture for Flash Sale Traffic Spikes

HACKATHON EVENT

SALESTORM Hackathon — High-Scale Engineering Track

SYSTEM CHALLENGE & FOCUS

Ultra-Low Latency Inventory Locking & Concurrency Control

TARGET SCALE & CAPACITY

10,000 Concurrent Requests / 100 Available Inventory Units

PROJECT TEAM MEMBERS

24AM301

Harisudharshan N

24AM304

Rohith M

24AM305

Srileka B

1. Project Overview & High-Level Design (HLD)

The **High-Scale E-Commerce Sale Architecture** is designed to manage extreme traffic bursts during high-demand flash sales. During the event, up to 10,000 customers hit the system concurrently within a single second to acquire only 100 available stock units.

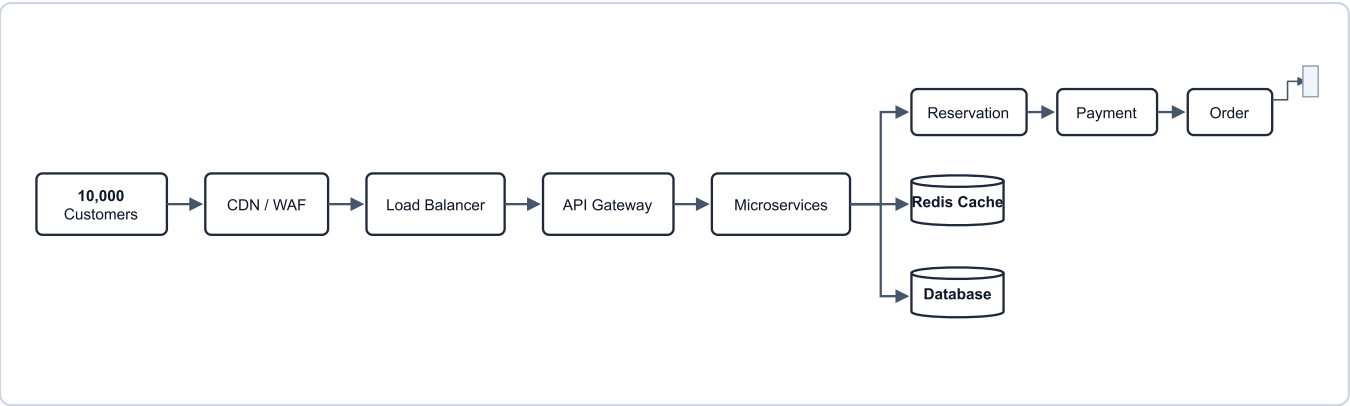
10,000
Concurrent Requests

100
Available Units

0%
Oversell Tolerance

High-Level System Architecture (Diagram 1)

The architecture isolates the high-traffic read/reservation path from the persistent write path. Global traffic passes through Edge CDN/WAF and Load Balancing before hitting Microservices backed by an in-memory Redis cluster for atomic inventory verification.

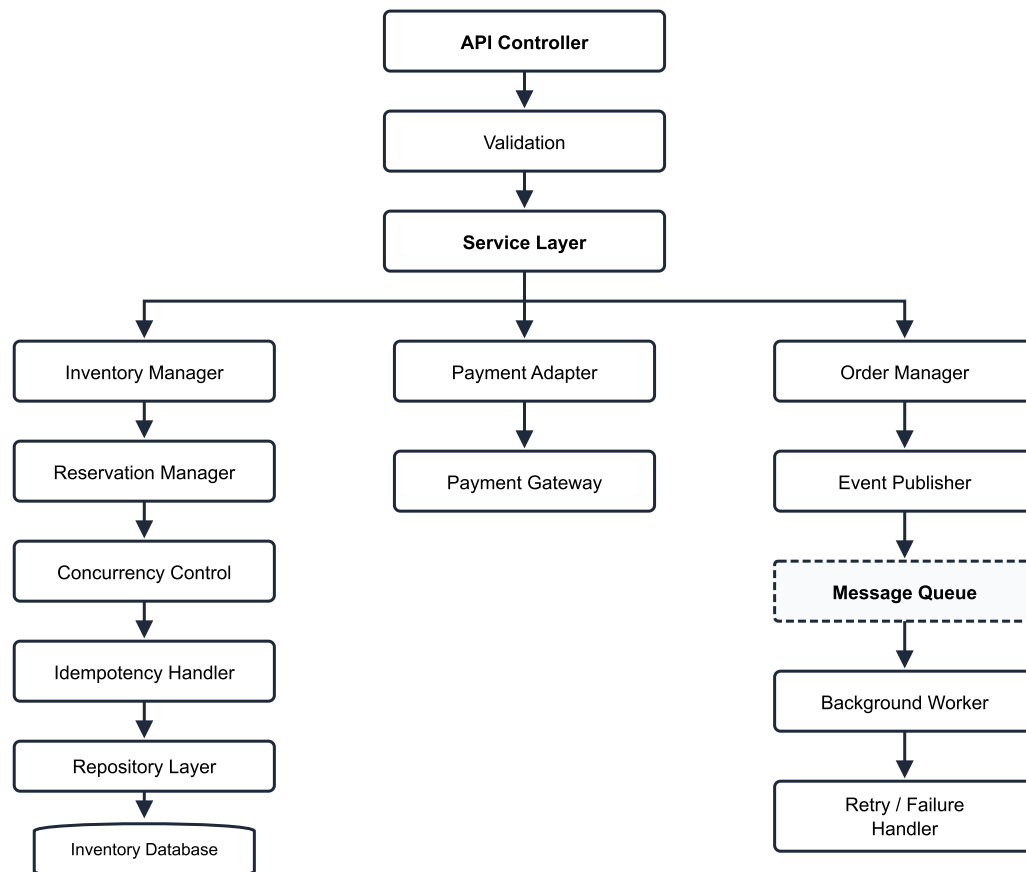


2. Low-Level Design (LLD)

The Low-Level Design defines the internal processing chain when an incoming purchase request hits the server. It handles validation, locks inventory, secures payment, and dispatches asynchronous background jobs.

Execution Path Integrity: The API Controller delegates work to specialized service layers. The inventory reservation path strictly verifies idempotency and concurrency locks prior to DB writes.

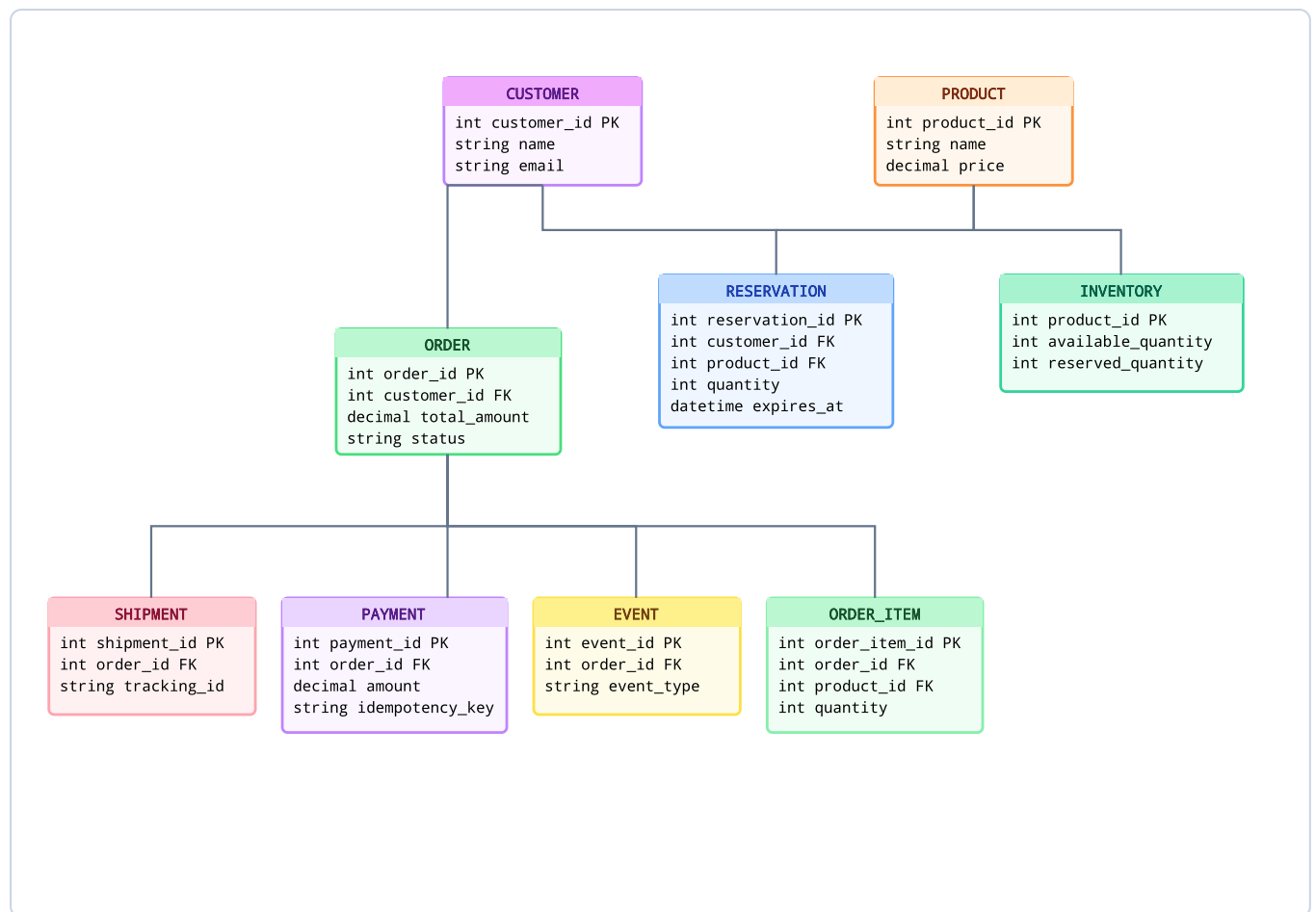
Detailed Layered Processing Diagram (Diagram 2)



3. Entity-Relationship & Database Design

The schema relies on high normalization for transactional integrity, isolating stock reservations from final paid orders to handle rapid temporary holds and auto-expirations seamlessly.

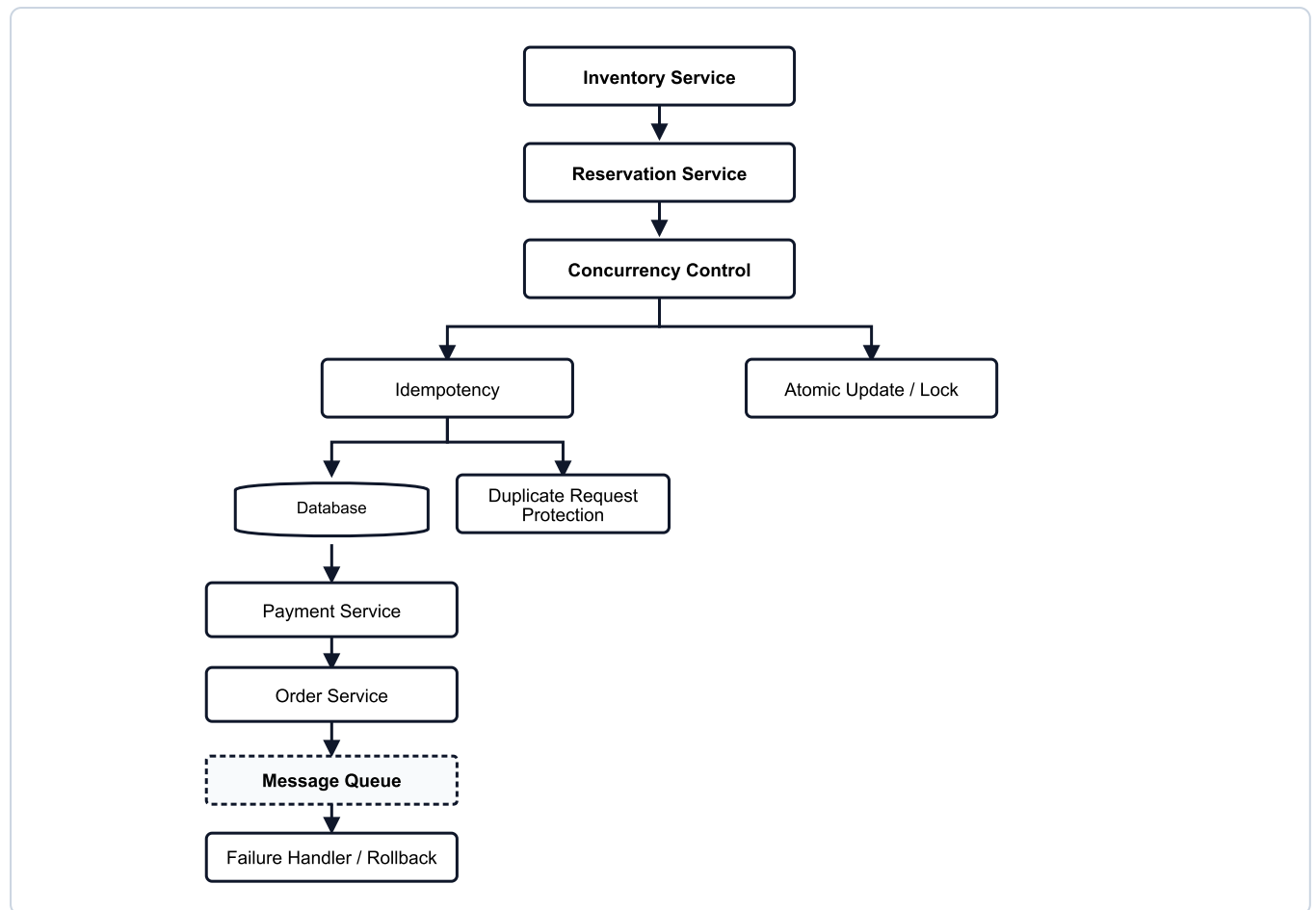
Entity Relationship Diagram (Diagram 3)



4. Component Architecture & Execution Mechanics

Diagram 4 illustrates how individual services isolate concurrency logic. Inventory and Reservation services execute atomic updates before interacting with Payment, Order, and asynchronous messaging components.

Inventory & Reservation Component Flow (Diagram 4)



5. Concurrency Handling, Scalability & Conclusion

Managing **10,000 concurrent requests against 100 inventory units** requires bypassing traditional SQL row locks. Standard locking leads to connection pool saturation, latency spikes, and cascading outages under high concurrency.

Preventing Overselling (Lua Scripting on Redis)

The system utilizes an atomic Lua script executed inside single-threaded Redis memory space to decrement stock and grant reservation keys deterministically.

```
local stock = tonumber(redis.call('get', KEYS[1])) if stock and stock > 0 then redis.call('decr',
```

Scalability & Reliability Framework

Layer	Mechanism	Failure Recovery / SLA
Ingress Rate Limiting	Token Bucket algorithm at API Gateway	Rejects excess traffic with 429 status (< 2ms)
Atomic Stock Locking	Redis Lua execution	Prevents race condition and overselling completely
Asynchronous Persistence	Kafka queue buffering order tasks	Dampens write load spike to relational DB
Reservation Expiry	Redis TTL key expiration webhook	Auto-restores inventory if payment fails

Conclusion

The **High-Scale E-Commerce Sale Architecture** achieves strict data consistency and zero-oversell operation under extreme flash-sale pressure. By leveraging decoupled asynchronous queues, in-memory atomic locks, and multi-tier rate limiting, the platform guarantees sub-10ms response times for 10,000 concurrent users while maintaining exact stock accuracy for the 100 available units.